OpenAI

Explain Bitmask DP

Yesterday 12:58 PM

Yes — this problem is a **classic Bitmask + Dynamic Programming problem**.

The most important observation is:

> We do **not** need to create a bitmask for people.
> We create a bitmask for the **required skills**.

Since:

```
req_skills.length <= 16
```

the total number of possible skill combinations is:

```
2^16 = 65536
```

That is small enough for DP.

---

1. Main idea

Suppose:

```
req_skills = ["java", "nodejs", "reactjs"]
```

Give every skill a bit position:

```
java     -> bit 0
nodejs   -> bit 1
reactjs  -> bit 2
```

So:

```
bit:      2       1       0
        react   node     java
```

The final state where all skills are covered is:

```
111₂
```

which is:

```
(1 << 3) - 1 = 7
```

So our goal is:

> Find the minimum number of people whose combined skill mask becomes `111`.

---

2. Convert every person into a bitmask

Example:

```
req_skills = ["java", "nodejs", "reactjs"]

people =
[
    ["java"],              // person 0
    ["nodejs"],            // person 1
    ["nodejs", "reactjs"]  // person 2
]
```

Person 0

Has:

```
java
```

Mask:

```
001
```

Person 1

Has:

```
nodejs
```

Mask:

```
010
```

Person 2

Has:

```
nodejs + reactjs
```

Mask:

```
110
```

So:

```
Person 0 → 001
Person 1 → 010
Person 2 → 110
```

3. What does `dp[mask]` mean?

We define:

```C++
dp[mask]
```

as:

> The smallest team of people required to obtain exactly the skills represented by `mask`.

For example:

```
mask = 001
```

means:

```
We have java
```

And:

```
mask = 111
```

means:

```
We have all required skills
```

The answer will be:

```C++
dp[(1 << n) - 1]
```

where:

```C++
n = req_skills.size()
```

4. Starting state

Initially, we have no skills:

```
000
```

and need no people:

```C++
dp[0] = {}
```

All other states are initially unavailable.

```
dp[000] = []
```

5. How do we add a person?

Suppose currently:

```
currentMask = 001
```

Meaning:

```
We have java
```

Now we add person 2:

```
personMask = 110
```

Use:

```C++
newMask = currentMask | personMask;
```

So:

```
    001
OR  110
--------
    111
```

Now all skills are covered.

This is exactly why bitwise OR is useful.

---

6. DP transition

For every person:

```C++
for(int mask = 0; mask < (1 << n); mask++)
```

we calculate:

```C++
newMask = mask | personMask[i];
```

Then:

> Can we reach `newMask` using a smaller team by adding this person?

So:

```C++
if(dp[newMask].size() > dp[mask].size() + 1)
```

then update:

```C++
dp[newMask] = dp[mask];
dp[newMask].push_back(i);
```

Full working on Example 1

Input

```
req_skills = ["java", "nodejs", "reactjs"]

people =
0 → ["java"]
1 → ["nodejs"]
2 → ["nodejs", "reactjs"]
```

Skill mapping:

```
java    → 0
nodejs  → 1
reactjs → 2
```

Masks:

```
Person 0 = 001
Person 1 = 010
Person 2 = 110
```

Initial DP:

```
dp[000] = []
```

Process Person 0

Person 0:

```
001
```

Starting from:

```
000
```

Add person 0:

```
000
OR
001
=
001
```

So:

```
dp[001] = [0]
```

Current states:

```
000 → []
001 → [0]
```

Process Person 1

Person 1:

```
010
```

From state `000`

```
000 | 010 = 010
```

So:

```
dp[010] = [1]
```

From state `001`

```
001 | 010 = 011
```

So:

```
dp[011] = [0,1]
```

Now:

```
000 → []
001 → [0]
010 → [1]
011 → [0,1]
```

Process Person 2

Person 2:

```
110
```

From `000`

```
000 | 110 = 110
```

```
dp[110] = [2]
```

From `001`

```
001 | 110
=
111
```

So:

```
dp[111] = [0,2]
```

This covers:

```
Person 0 → java
Person 2 → nodejs + reactjs
```

Therefore:

```
java + nodejs + reactjs
```

All covered with only:

```
[0, 2]
```

Visual representation

```
Required Skills

java     nodejs     reactjs
 ↓          ↓          ↓
bit 0      bit 1      bit 2


Final target:

java nodejs reactjs
 1      1       1

        111
```

People:

```
Person 0

java
 ↓

001
```

```
Person 1

nodejs
 ↓

010
```

```
Person 2

nodejs + reactjs

010 + 100 = 110
```

Combining Person 0 and Person 2:

```
Person 0        Person 2

   001       |     110

      ↓ OR ↓

          111
```

Therefore:

```
Team = [0, 2]
```

The complete DP state graph

```
                         Person 2
                      +-------------+
                      |             |
                      v             |
                  110 → [2]         |
                                    |
000 → []                            |
 |
 | Person 0                         |
 v                                  |
001 → [0] ------ Person 2 --------> 111 → [0,2]
 |                                     ↑
 |
 | Person 1                           |
 v                                     |
011 → [0,1]                            |
                                       |
000 ---------- Person 1 -----------> 010 → [1]
```

The important transition is always:

```C++
newMask = mask | personMask[i];
```

C++ Code

```C++
class Solution
{
public:
    vector<int> smallestSufficientTeam(vector<string>& req_skills,
                                       vector<vector<string>>& people)
    {
        int n = req_skills.size();
        int m = people.size();

        // Map every skill to a bit position
        unordered_map<string, int> skillIndex;

        for(int i = 0; i < n; i++)
        {
            skillIndex[req_skills[i]] = i;
        }

        // Convert every person's skills into a bitmask
        vector<int> personMask(m, 0);

        for(int i = 0; i < m; i++)
        {
            for(string& skill : people[i])
            {
                int bit = skillIndex[skill];

                personMask[i] |= (1 << bit);
            }
        }

        // dp[mask] = smallest team that can cover 'mask'
        vector<vector<int>> dp(1 << n);

        // Mark unreachable states
        for(int mask = 1; mask < (1 << n); mask++)
        {
            dp[mask] = vector<int>(100);
        }

        // dp[0] = empty team
        dp[0] = {};

        // Try adding every person
        for(int i = 0; i < m; i++)
        {
            // Traverse all possible skill states
            for(int mask = 0; mask < (1 << n); mask++)
            {
                // Skip unreachable states
                if(dp[mask].size() == 100)
                    continue;

                // Add current person's skills
                int newMask = mask | personMask[i];

                // If this creates a smaller team
                if(dp[newMask].size() > dp[mask].size() + 1)
                {
                    dp[newMask] = dp[mask];
                    dp[newMask].push_back(i);
                }
            }
        }

        // Full skill mask
        int fullMask = (1 << n) - 1;
```

```
        return dp[fullMask];
    }
};
```

A slightly cleaner version

Instead of using:

```C++
vector<int>(100)
```

we can use an **INF**.

```C++
class Solution
{
public:
    vector<int> smallestSufficientTeam(vector<string>& req_skills,
                                       vector<vector<string>>& people)
    {
        int n = req_skills.size();
        int m = people.size();

        unordered_map<string, int> skillIndex;

        // Assign every required skill a bit
        for(int i = 0; i < n; i++)
        {
            skillIndex[req_skills[i]] = i;
        }

        // Convert every person into a skill mask
        vector<int> personMask(m);

        for(int i = 0; i < m; i++)
        {
            for(string skill : people[i])
            {
                personMask[i] |= (1 << skillIndex[skill]);
            }
        }

        // dp[mask] = minimum team that covers this skill mask
        vector<vector<int>> dp(1 << n);

        // Empty vector means unreachable initially
        vector<bool> reachable(1 << n, false);

        reachable[0] = true;

        // Process every person
        for(int i = 0; i < m; i++)
        {
            for(int mask = 0; mask < (1 << n); mask++)
            {
                if(!reachable[mask])
                    continue;

                int newMask = mask | personMask[i];

                // Current team + this person
                if(!reachable[newMask] ||
                   dp[newMask].size() > dp[mask].size() + 1)
                {
                    dp[newMask] = dp[mask];
                    dp[newMask].push_back(i);

                    reachable[newMask] = true;
                }
            }
        }

        return dp[(1 << n) - 1];
    }
};
```

Why are we doing Bitmasking on skills instead of people?

This is extremely important.

We have:

```
People ≤ 60
Skills ≤ 16
```

If we create a bitmask for people:

```
2^60
```

Impossible ❌

But if we create a bitmask for skills:

```
2^16 = 65536
```

Possible ✅

So we always choose the smaller dimension for the DP state.

```
People:
60
↓
2^60 ❌

Skills:
16
↓
2^16 = 65536 ✅
```

That is the key insight of this problem.

Time Complexity

Let:

```
N = number of required skills
M = number of people
```

We have:

```
N ≤ 16
M ≤ 60
```

For every person, we iterate through every mask:

```
O(M × 2^N)
```

Since:

```
60 × 65536
≈ 4 million
```

which is manageable.

Technically, because we copy a team vector during updates, the practical complexity includes the team length, but the maximum team size is small enough for these constraints.

The whole logic in one picture

```
STEP 1
────────────────────────

req_skills

["java", "nodejs", "reactjs"]

       ↓ assign bits

java      → 001
nodejs    → 010
reactjs   → 100

STEP 2
────────────────────────

Convert people into masks

Person 0 → ["java"]
             ↓
            001

Person 1 → ["nodejs"]
             ↓
            010

Person 2 → ["nodejs", "reactjs"]
             ↓
            110

STEP 3
────────────────────────

DP State

dp[mask]

mask = skills currently covered
```

```
000 → no skills
001 → java
010 → nodejs
011 → java + nodejs
100 → reactjs
...
111 → all skills

STEP 4
──────────────────

For every person:

newMask = currentMask | personMask


Example:

currentMask = 001

personMask  = 110
                 OR
              ───────

                111

STEP 5
──────────────────

Keep the smallest team:

if(new team size < old team size)
{
    update dp[newMask]
}

STEP 6
──────────────────

Return:

dp[111]
```

Core formula to remember

```C++
// State = skills currently covered
dp[mask]

// Add a person's skills
newMask = mask | personMask[i]

// Minimize number of people
if(dp[newMask].size() > dp[mask].size() + 1)
{
    dp[newMask] = dp[mask];
    dp[newMask].push_back(i);
}
```

This problem is essentially **0/1 DP over subsets**:

> **State = subset of required skills covered**
> **Choice = take or don't take each person**
> **Transition = bitwise OR with that person's skill mask**
> **Goal = reach `(1 << n) - 1` with the minimum number of people**.

Yes. This part is **converting every person's list of skills into a single integer bitmask**.

Let's take the code slowly:

```C++
for(int i = 0; i < m; i++)
{
    for(string& skill : people[i])
    {
        int bit = skillIndex[skill];

        personMask[i] |= (1 << bit);
    }
}
```

Suppose we have:

```C++
req_skills = ["java", "nodejs", "reactjs"];
```

First, we create this mapping:

```C++
skillIndex["java"] = 0;
skillIndex["nodejs"] = 1;
skillIndex["reactjs"] = 2;
```

So internally:

```
Skill        Index / Bit Position

java              0
nodejs            1
reactjs           2
```

Step 1: `people[i]`

Suppose:

```C++
people = {
    {"java"},
    {"nodejs"},
    {"nodejs", "reactjs"}
};
```

We want to convert this into:

```
Person 0 → 001
Person 1 → 010
Person 2 → 110
```

Let's see how.

Outer loop

```C++
for(int i = 0; i < m; i++)
```

This goes through every person.

If:

```
m = 3
```

then:

```
i = 0 → Person 0
i = 1 → Person 1
i = 2 → Person 2
```

Person 0

```C++
i = 0
```

Their skills are:

```C++
people[0] = {"java"};
```

Now this loop:

```C++
for(string& skill : people[i])
```

means:

> Go through every skill that person `i` has.

So here:

```C++
skill = "java";
```

This line

```C++
int bit = skillIndex[skill];
```

We have:

```C++
skill = "java"
```

Therefore:

```C++
skillIndex["java"] = 0;
```

So:

```C++
bit = 0;
```

Now we execute:

```C++
personMask[i] |= (1 << bit);
```

Since:

```
bit = 0
```

we get:

```C++
1 << 0
```

Binary representation:

```
1 << 0

0001
```

So:

```C++
personMask[0] |= 0001;
```

Initially:

```
personMask[0] = 000
```

After OR:

```
000
OR
001
---
001
```

Therefore:

```
Person 0 = 001
```

Person 1

```C++
i = 1
```

```C++
people[1] = {"nodejs"};
```

The inner loop gives:

```C++
skill = "nodejs";
```

Then:

```C++
int bit = skillIndex["nodejs"];
```

From our mapping:

```
nodejs → 1
```

So:

```C++
bit = 1;
```

Now:

```C++
1 << bit
```

becomes:

```C++
1 << 1
```

Which is:

```
0010
```

Then:

```C++
personMask[1] |= 0010;
```

Initially:

```
000
```

After:

```
000
OR
010
---
010
```

Therefore:

```
Person 1 = 010
```

Person 2 — the important one

```C++
people[2] = {"nodejs", "reactjs"};
```

Initially:

```
personMask[2] = 000
```

The inner loop runs **twice**.

First skill: `"nodejs"`

```C++
skill = "nodejs"
```

```C++
bit = skillIndex["nodejs"];
```

So:

```
bit = 1
```

Then:

```C++
1 << 1 = 010
```

Update:

```C++
personMask[2] |= 010;
```

Initially:

```
000
```

So:

```
000
OR
010
---
010
```

Now:

```
personMask[2] = 010
```

Second skill: `"reactjs"`

Now:

```C++
skill = "reactjs";
```

From the map:

```
reactjs → 2
```

Therefore:

```C++
bit = 2;
```

Then:

```C++
1 << 2
```

becomes:

```
0100
```

or using 3 bits:

```
100
```

Now:

```C++
personMask[2] |= 100;
```

Current value:

```
010
```

New skill:

```
100
```

Apply OR:

```
010
OR
100
```

```
---
110
```

So finally:

```
personMask[2] = 110
```

Complete dry run

```
Required Skills:

java      → bit 0
nodejs    → bit 1
reactjs   → bit 2
```

Person 0

```
Skills: ["java"]

personMask[0] = 000

java
 ↓
bit = 0

1 << 0 = 001

000 | 001 = 001
```

Final:

```
Person 0 → 001
```

Person 1

```
Skills: ["nodejs"]

personMask[1] = 000

nodejs
 ↓
bit = 1

1 << 1 = 010

000 | 010 = 010
```

Final:

```
Person 1 → 010
```

Person 2

```
Skills: ["nodejs", "reactjs"]

personMask[2] = 000
```

First skill:

```
nodejs

bit = 1

1 << 1 = 010

000 | 010 = 010
```

Second skill:

```
reactjs

bit = 2

1 << 2 = 100

010 | 100 = 110
```

Final:

```
Person 2 → 110
```

Final `personMask` array

```
Index          Binary       Meaning
0               001         java
1               010         nodejs
2               110         nodejs + reactjs
```

So:

```C++
personMask = {1, 2, 6};
```

Because:

```
001 = 1
010 = 2
110 = 6
```

Why do we use `|=`?

This line:

```C++
personMask[i] |= (1 << bit);
```

is equivalent to:

```C++
personMask[i] = personMask[i] | (1 << bit);
```

We use `|` because one person can have **multiple skills**.

For example:

```
Current person skills:

nodejs = 010
reactjs = 100
```

Combining them:

```
010
OR
100
---
110
```

So the final mask stores **all skills of that person in one number**.

The whole purpose of this code is:

```
["nodejs", "reactjs"]

        ↓

      110
```

Then later, the DP can simply combine a person's skills using:

```C++
newMask = currentMask | personMask[i];
```

So instead of working with strings like:

```
"java"
"nodejs"
"reactjs"
```

again and again during DP, we convert them **once** into binary masks, making the DP fast and simple.

Yes — this part is used to initialize the **DP array** and mark which skill combinations are currently **reachable** or **unreachable**.

Let's break it down carefully.

```C++
// dp[mask] = smallest team that can cover 'mask'
vector<vector<int>> dp(1 << n);
```

1. What is `dp`?

```C++
vector<vector<int>> dp(1 << n);
```

Suppose:

```C++
n = 3;
```

because we have 3 required skills:

```
java
nodejs
reactjs
```

Then:

```C++
1 << n
```

means:

```
1 << 3 = 8
```

So:

```C++
vector<vector<int>> dp(8);
```

This creates 8 states:

```
dp[0]
dp[1]
dp[2]
dp[3]
dp[4]
dp[5]
dp[6]
dp[7]
```

These represent all possible combinations of 3 skills.

---

2. What does each `dp[mask]` store?

Each mask represents a set of skills.

```
Mask        Skills

000         no skills
001         java
010         nodejs
011         java + nodejs
100         reactjs
101         java + reactjs
110         nodejs + reactjs
111         all skills
```

And:

```C++
dp[mask]
```

stores the **team of people** that covers that skill combination.

For example:

```
dp[001] = [0]
```

means:

> Person `0` covers the skills represented by `001`.

If person `0` has:

```
["java"]
```

then:

```
dp[001] = [0]
```

Similarly:

```
dp[110] = [2]
```

means person `2` alone covers:

```
nodejs + reactjs
```

3. Why is it `vector<vector<int>>`?

Because each DP state stores a **team**, and a team is a list of person indices.

For example:

```
dp[111] = [0, 2]
```

So:

```C++
vector<int>
```

stores one team.

And we need this for every possible mask:

```C++
vector<vector<int>> dp;
```

You can visualize it like:

```
dp

dp[000] → []
dp[001] → [0]
dp[010] → [1]
dp[011] → [0,1]
dp[100] → ?
dp[101] → ?
dp[110] → [2]
dp[111] → [0,2]
```

4. What happens initially?

When we write:

```C++
vector<vector<int>> dp(1 << n);
```

every vector is initially an **empty vector**.

So initially:

```
dp[000] = []
dp[001] = []
dp[010] = []
dp[011] = []
dp[100] = []
dp[101] = []
dp[110] = []
dp[111] = []
```

But there is a problem.

We need to distinguish between:

```
[]
```

meaning:

> No team is required.

and:

```
[]
```

meaning:

> This state has not been reached yet.

Both look exactly the same!

5. Why do we mark unreachable states with `vector<int>(100)`?

This code:

```C++
for(int mask = 1; mask < (1 << n); mask++)
{
    dp[mask] = vector<int>(100);
}
```

starts from:

```
mask = 1
```

and goes up to:

```
(1 << n) - 1
```

For every state except `0`, we store a vector containing 100 elements.

So initially:

```
dp[000] = []
```

but:

```
dp[001] = [x,x,x,x,x,...]  // size = 100
dp[010] = [x,x,x,x,x,...]  // size = 100
dp[011] = [x,x,x,x,x,...]  // size = 100
...
```

We don't care about the actual values inside.

The important thing is:

```C++
dp[mask].size() == 100
```

means:

> This state is currently unreachable.

---

6. Why specifically `100`?

Because:

```
number of people <= 60
```

So a valid team can never have 100 people.

The maximum possible valid team size is at most:

```
60
```

Therefore, size `100` is acting like:

```
INF
```

or:

> A very large value.

So:

```
Team size = 100
```

basically means:

```
∞
```

or:

```
unreachable
```

7. What is happening with this line?

```C++
dp[0] = {};
```

Mask `0` means:

```
000
```

For 3 skills:

```
java = 0
nodejs = 0
reactjs = 0
```

So we currently have:

```
No skills.
```

What people do we need to have no skills?

```
Nobody.
```

Therefore:

```C++
dp[0] = {};
```

means:

```
dp[000] = empty team
```

This is our **starting point**.

Initial DP looks like this

Suppose:

```
n = 3
```

Then:

```
Mask        dp[mask]

000         []          ← reachable, starting state

001         size 100    ← unreachable
010         size 100    ← unreachable
011         size 100    ← unreachable
100         size 100    ← unreachable
101         size 100    ← unreachable
110         size 100    ← unreachable
111         size 100    ← unreachable
```

Now let's see why this is needed

Suppose we process:

```
Person 0 = java = 001
```

We start with:

```
mask = 000
```

And:

```C++
newMask = mask | personMask[i];
```

So:

```
000
OR
001
```

```
---
001
```

Now we check:

```C++
if(dp[newMask].size() > dp[mask].size() + 1)
```

Substitute the values:

```
dp[001].size() = 100
dp[000].size() = 0
```

Therefore:

```
100 > 0 + 1
```

```
100 > 1
```

True ✅

So:

```C++
dp[001] = dp[000];
```

Currently:

```
dp[000] = []
```

Therefore:

```
dp[001] = []
```

Then:

```C++
dp[001].push_back(0);
```

Now:

```
dp[001] = [0]
```

So the state changes:

```
Before:

001 → size 100 ❌ unreachable

After adding Person 0:

001 → [0] ✅
```

The important idea

This:

```C++
vector<vector<int>> dp(1 << n);
```

creates all possible skill states.

This:

```C++
for(int mask = 1; mask < (1 << n); mask++)
{
    dp[mask] = vector<int>(100);
}
```

marks every state as initially:

```
UNREACHABLE
```

And:

```C++
dp[0] = {};
```

marks:

```
NO SKILLS
```

as our starting state.

In one picture

```
                    START

                      ↓

                  dp[000]
                     []

              No skills covered
              No people required


                      │ Add Person 0
                      │
                      ▼

                  dp[001]
                     [0]

                 java covered


                      │ Add Person 2
                      │
                      ▼

                  dp[111]
                     [0,2]

              All skills covered
```

So the DP starts from:

```C++
dp[0] = {};
```

and gradually builds other states by adding people.

The final answer is:

```C++
dp[(1 << n) - 1]
```

which represents:

```
111111...111
```

meaning:

> **All required skills are covered, and `dp` stores the smallest team that achieves it.**

Yes. This is the **main DP transition**, and this is where the actual logic of the solution happens.

We will dry run **exactly this code** on:

```C++
req_skills = ["java", "nodejs", "reactjs"]

people = [
    ["java"],             // Person 0
    ["nodejs"],           // Person 1
    ["nodejs", "reactjs"]  // Person 2
]
```

1. Before this loop starts

We already converted every person into a bitmask:

```
Person 0 → ["java"]              → 001
Person 1 → ["nodejs"]            → 010
Person 2 → ["nodejs","reactjs"]  → 110
```

And our DP initially is:

```
Mask        dp[mask]

000         []              reachable
001         size 100        unreachable
010         size 100        unreachable
011         size 100        unreachable
100         size 100        unreachable
101         size 100        unreachable
110         size 100        unreachable
111         size 100        unreachable
```

Remember:

```
dp[mask] = minimum team that covers mask
```

2. The main loop

```C++
for(int i = 0; i < m; i++)
{
    for(int mask = 0; mask < (1 << n); mask++)
    {
        if(dp[mask].size() == 100)
            continue;

        int newMask = mask | personMask[i];

        if(dp[newMask].size() > dp[mask].size() + 1)
        {
            dp[newMask] = dp[mask];
            dp[newMask].push_back(i);
        }
    }
}
```

The logic is:

> Take every person one by one.
> From every currently reachable skill combination, try adding that person.

---

3. Process Person 0

Outer loop:

```C++
i = 0
```

Person 0:

```
personMask[0] = 001
```

Person 0 has only:

```
java
```

`mask = 000`

We check:

```C++
if(dp[000].size() == 100)
```

But:

```
dp[000] = []
size = 0
```

So this state is reachable.

Now:

```C++
int newMask = mask | personMask[i];
```

Substitute:

```
mask          = 000
personMask[0] = 001
```

So:

```
000
OR
001
---
001
```

Therefore:

```C++
newMask = 001;
```

Now check:

```C++
if(dp[newMask].size() > dp[mask].size() + 1)
```

Which means:

```
dp[001].size() > dp[000].size() + 1
```

Currently:

```
100 > 0 + 1
100 > 1
```

True ✅

So update:

```C++
dp[001] = dp[000];
```

Currently:

```
dp[000] = []
```

Therefore:

```
dp[001] = []
```

Then:

```C++
dp[001].push_back(0);
```

Now:

```
dp[001] = [0]
```

Meaning:

> To get skill mask **`001`**, use Person 0.

DP now:

```
000 → []
001 → [0]
010 → unreachable
011 → unreachable
100 → unreachable
101 → unreachable
110 → unreachable
111 → unreachable
```

Now `mask = 001`

This state is now reachable:

```
dp[001] = [0]
```

Now calculate:

```
001
OR
001
---
001
```

So:

```C++
newMask = 001;
```

This means:

> If we already have Java and add Person 0 again, we still only have Java.

Now the condition becomes:

```
dp[001].size() > dp[001].size() + 1

1 > 1 + 1

1 > 2
```

False ❌

So we don't update.

This prevents adding the same person unnecessarily.

All other masks are unreachable:

```
010 → size 100
011 → size 100
100 → size 100
...
```

So:

```C++
if(dp[mask].size() == 100)
    continue;
```

skips them.

DP after Person 0

```
Mask        Team

000         []
001         [0]
010         unreachable
011         unreachable
100         unreachable
101         unreachable
110         unreachable
111         unreachable
```

4. Process Person 1

Now:

```C++
i = 1;
```

Person 1:

```
personMask[1] = 010
```

Meaning:

```
nodejs
```

`mask = 000`

We have:

```
dp[000] = []
```

Reachable.

Calculate:

```
000
OR
010
---
010
```

So:

```C++
newMask = 010;
```

Check:

```
dp[010].size() > dp[000].size() + 1

100 > 0 + 1

100 > 1
```

True.

Update:

```
dp[010] = [1]
```

Now:

```
000 → []
001 → [0]
010 → [1]
```

`mask = 001`

Currently:

```
dp[001] = [0]
```

Meaning:

```
We already have:

java
```

Now add Person 1:

```
001
OR
010
---
011
```

So:

```C++
newMask = 011;
```

Meaning:

```
java + nodejs
```

Now compare:

```
dp[011].size() > dp[001].size() + 1

100 > 1 + 1

100 > 2
```

True.

So:

```C++
dp[011] = dp[001];
```

Thus:

```
dp[011] = [0]
```

Then:

```C++
dp[011].push_back(1);
```

So:

```
dp[011] = [0,1]
```

Meaning:

```
Person 0 → java
Person 1 → nodejs
```

Together:

```
001
OR
010
---
011
```

`mask = 010`

Currently:

```
dp[010] = [1]
```

Now add Person 1 again:

```
010
OR
010
---
010
```

No new skills.

New team would be:

```
[1,1]
```

Size 2.

But existing:

```
[1]
```

Size 1.

Condition:

```
1 > 2
```

False.

So we keep:

```
dp[010] = [1]
```

DP after Person 1

```
000 → []
001 → [0]
010 → [1]
011 → [0,1]
100 → unreachable
101 → unreachable
110 → unreachable
111 → unreachable
```

5. Process Person 2

Now:

```C++
i = 2;
```

Person 2:

```
personMask[2] = 110
```

Meaning:

```
nodejs + reactjs
```

This is where we reach the final answer.

`mask = 000`

Currently:

```
dp[000] = []
```

Add Person 2:

```
000
OR
110
---
110
```

So:

```C++
newMask = 110;
```

Check:

```
100 > 0 + 1
```

True.

Update:

```
dp[110] = [2]
```

Now:

```
110 → [2]
```

`mask = 001`

Currently:

```
dp[001] = [0]
```

Meaning:

```
Person 0 covers:

001 → java
```

Now add Person 2:

```
001
OR
110
---
111
```

Boom 💥

Now:

```
newMask = 111
```

which means:

```
java + nodejs + reactjs
```

Now compare:

```
dp[111].size() > dp[001].size() + 1
```

Currently:

```
100 > 1 + 1

100 > 2
```

True.

So:

```C++
dp[111] = dp[001];
```

Therefore:

```
dp[111] = [0]
```

Then:

```C++
dp[111].push_back(2);
```

Finally:

```
dp[111] = [0,2]
```

This is the key transition

```
Current state:

001 → [0]

Person 2:

110


        001
         |
         | Add Person 2
         ↓
        OR
         ↓
        110

    001 | 110

        ↓

       111
```

And the team becomes:

```
[0] + [2]

↓

[0,2]
```

So:

```
dp[111] = [0,2]
```

Now understand the most important logic

This code:

```C++
int newMask = mask | personMask[i];
```

asks:

> If I currently have the skills in `mask`, what skills will I have after adding person `i`?

Example:

```
mask = 001

We have:

java
```

Person 2:

```
110

nodejs + reactjs
```

Combine:

```
001
OR
110
---
111
```

Now we have:

```
java + nodejs + reactjs
```

Why do we compare sizes?

This:

```C++
if(dp[newMask].size() > dp[mask].size() + 1)
```

means:

> Is the old team for **newMask** bigger than the new team I just created?

For example:

Suppose:

```
dp[111] = [0,1,3]
```

Size:

```
3
```

But from:

```
dp[001] = [0]
```

we add Person 2:

```
[0] + [2]

↓

[0,2]
```

Size:

```
2
```

So:

```
Old solution = 3 people
New solution = 2 people
```

Therefore update.

Full DP flow visually

```
START

dp[000] = []
     |
     |
     | Add Person 0 (001)
     ▼

dp[001] = [0]
     |
```

```
      |
      | Add Person 1 (010)
      ▼

dp[011] = [0,1]


Meanwhile:

dp[000] = []
      |
      |
      | Add Person 2 (110)
      ▼

dp[110] = [2]


Most important:

dp[001] = [0]
      |
      |
      | Add Person 2 (110)
      ▼

001 OR 110
      |
      ▼

111

dp[111] = [0,2]
```

What does this loop fundamentally do?

```C++
for(int i = 0; i < m; i++)
```

Means:

> Consider every person.

```C++
for(int mask = 0; mask < (1 << n); mask++)
```

Means:

> For this person, try adding them to every skill combination we have already reached.

```C++
if(dp[mask].size() == 100)
    continue;
```

Means:

> If we cannot reach this skill combination, don't use it.

```C++
int newMask = mask | personMask[i];
```

Means:

> Add this person's skills to our current skills.

```C++
if(dp[newMask].size() > dp[mask].size() + 1)
```

Means:

> If adding this person gives us a smaller team, update the answer for this skill combination.

```C++
dp[newMask] = dp[mask];
dp[newMask].push_back(i);
```

Means:

> Copy the old team and add the current person.

The entire logic in one sentence

```
For every person,
    try adding them to every reachable combination of skills,
    and for every resulting skill combination,
    keep only the smallest team.
```

Finally:

```C++
int fullMask = (1 << n) - 1;
```

For 3 skills:

```
1 << 3 = 1000

1000 - 1 = 0111

= 111
```

So:

```
fullMask = 111
```

means:

> Every required skill is covered.

Therefore:

```C++
return dp[fullMask];
```

returns:

```
dp[111]

↓

[0,2]
```

which is the smallest sufficient team.